摘　要

本文从图论的基本知识出发，以建造成本尽可能低和用户用电可靠性尽可能高为优化目标，通过聚类算法将题目**分解成主线和支线两个问题**分别求最优解，来确定在一定范围内的配电网规划方法。

对于问题一，将线路分为两级，分别为主线和支线。每一个支线由几个用户构成，他们共同连在其中一个点上并入主线。首先通过 **K-means 聚类算法**将用户分为若干个类，每一个类代表一个支线。随后求出每一类的中心点作为与主线相连的点。对于主线规划问题，利用两种不同的方法进行操作，并以**线性回归的 R 值**为标准在两种方法中进行选择。方法一是利用**最小二乘法**寻找一条直线使得所有点到该直线的距离的平方和最小；方法二是**使用模拟退火求解单向 TSP 问题**，寻找一条经过电源和每类中心点一次且长度最小的线路。对于支线部分的线路求解，使用**斯坦纳树模型**进行求解，来保证支线长度尽可能短。

对于问题二，与问题一类似，在问题一的基础上将线路主线分为两段，每段分别包含一个电源，并且使得两段线路长度总和尽可能短。

对于问题三，需要在问题二的求解结果的基础上，构造添加电线的策略。本文将每段电线内的部分简化为一个电路单元，采用**贪心算法**，首先根据最小的若干个用户用电可靠性之间的差值来判断当前步骤需要综合考虑几个用户，然后对需要考虑区域内的电路单元进行遍历，依次求解在另一个单供配电网上的**最优添线位置**，得到该步的最优添线策略，以此循环，直到经费达到上限。

对于问题四，与问题三类似，在问题三模型的基础上，将循环终止条件改为所有节点的可靠性均大于 Y%，求出此时的经费即为最优花费。

关键词　K-means 聚类　单向 TSP 问题　模拟退火　斯坦纳树模型　贪心算法

目录

1 问题的提出和重述 **3**

2 符号说明及定义说明 **3**

2.1 符号说明 3

3 模型的整体假设 **3**

3.1 开关的假设 3

3.2 数据生成策略 4

3.2.1 坐标生成策略 4

3.2.2 需求功率生成策略 5

3.3 用户用电可靠性计算模型 5

4 问题一与问题二 **6**

4.1 模型的假设 6

4.2 模型的求解 7

4.2.1 主线的求解-聚类与 TSP 模型求解 7

4.2.2 主线的求解-最小二乘法 9

4.2.3 支线的求解 9

4.2.4 问题一求解结果 10

4.2.5 问题二求解结果 11

5 问题三与问题四 **13**

5.1 模型的假设 13

5.1.1 用电可靠性更新模型 13

5.1.2 基于贪婪思想的每轮需考虑用户的确定方式 13

5.2 求解结果 15

6 参考文献 **17**

7 附录 **18**

7.1 pro1.py 18

1 问题的提出和重述

本题背景为配电网络的线路规划问题，基于题意，本题忽略电能在运输过程中的损耗等影响因素，着重考虑用户的用电可靠性，即内不因配电网故障停电或限电的概率，以及建设价格问题。其中，线路的设计需要满足题述的三条原则。

对于问题一，需要在给定一个电源、若干个用户的坐标，每个用户的需求功率的条件下，设计一个满足设计原则的单供配电网络，以建设价格最低为唯一的优化目标。

对于问题二，需要在给定两个电源坐标、若干个用户的坐标、每个用户的需求功率的条件下，设计两个满足设计原则的单供配电网络，同样以建设价格最低为唯一的优化目标。

对于问题三，需要基于问题二的两个单供配电网络，添加若干电线与开关，建设成一个双供配电网络。该双供配电网络需要在满足总花费小于 X 的约束条件下，以最低用户用电可靠性最大化为优化目标。

对于问题四，同样基于问题二的两个单供配电网络，添加若干电线与开关，建设成一个双供配电网络。该双供配电网络需要在满足最低用户用电可靠性大于 Y 的约束条件下，以总花费最小化为优化目标。

2 符号说明及定义说明

2.1 符号说明

- 对数正态分布 $N'(\mu, \sigma^2)$
- 主线长度 L_m
- 支线 A 长度 L_a
- 支线 B 长度 L_b
- 支线开关数量 S_1
- 主线开关数量 S_1
- 用电可靠性 R

3 模型的整体假设

3.1 开关的假设

由题目所述，开关设置原则如下：

1. 配电网中开关（电源出线后开关除外）的设置必须使得网中某处发生故障时，通过开关隔离故障后，保持供电的用户需求功率之和最大化；

2. 在网上任意一点到电源的所有路中，可以通过设置开关状态使得仅有一条是通路；

3. 配电网中开关的设置包含但不限于以下情况：每个用户前端有开关，每个分岔点后端的每条支路上有开关，双供配电网的每条联络线上有开关。

基于上述原则，根据原则 3 可知，电网中支线开关的数量等于用户数量。根据原则 2，电网不存在无开关的闭合环路，又根据建设价格的优化目标，本文进一步作出电路无环路的假设。根据原则 1，本文将其等价于每个用户到主线有且仅有一个开关，即不存在用户之间串联的关系。

3.2 数据生成策略

3.2.1 坐标生成策略

在构造各负荷的 x、y 坐标时，需要考虑在实际的配电网络应用中个需求端的地理位置分布。不妨以居民小区为例，各个需求端的地理位置分布较为均匀。为力更方便且有依据的生成数据集，本文采用的检验数据集均假设需求端的 x、y 坐标服从参数分布族，具体采用了网格状分布与 x、y 坐标分别服从两个独立的均匀分布两种情形。接着，为确定两个分布族的参数，需确定线路边长的期望。本文从故障单元故障率的角度出发，由题目可知，每个电源和每个用户的故障率都是 0.5%；每个开关的故障率均是 0.2%，两者接近，于是每条线路的故障率存在三种情况：与这两者故障率接近、远小于、远大于。本文对这三种情况分别进行了假设。

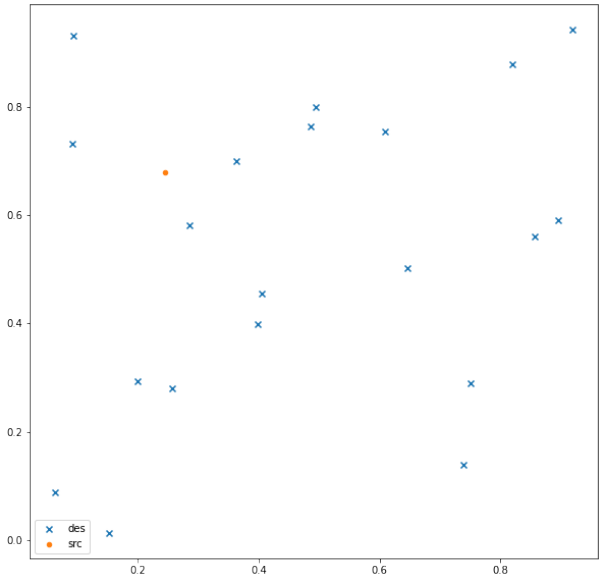

图 1: 随机生成用户坐标结果

3.2.2 需求功率生成策略

从实际情况出发，用户用电需求功率分布基本符合正态分布的分布函数。但正态分布的定义域在全体实数集上，即会出现负数，而需求功率的值域为 $[0, +\infty\square$，故考虑函数 $f(x) = e^x$，将需求功率假设为对数正态分布。对数正态分布的概率密度函数如下图所示，发现与正态分布相比，对数正态分布的密度函数不为对称，极大值点左偏，在趋于正无穷的区域密度函数更小，即对应实际情况中极端大的用电功率的出现概率比较小值出现概率更低，与实际情况更为符合。故对数正态分布的假设是与实际相符的，记作 $X\ N'(\mu, \sigma^2)$。

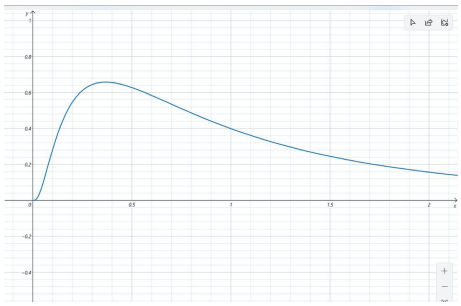

图 2: 对数正态分布的概率密度函数

同时由于本题关于功率的限定均以比例作为衡量指标，故 $\mu = EX$, 对整体的运算结果不产生影响。而标示偏离度的参数 σ 并没有先验信息，故首先从标准正态分布衍生出的对数正态分布考虑，即令 $\mu = 0, \sigma = 1$.

Algorithm 1 produce X $N'(\mu, \sigma^2)$

```
Initialization:μ = 0, σ = 1
for i ← 2 to n do
    Y[i] = random_number()
    %use packages to produce random numbers in normalized distribution
    X[i] = exp(Y[i])
end for
```

Output: X[1:n]

3.3 用户用电可靠性计算模型

为了简化添线时的位置选择，本文将两个节点直接的部分视作一个单元（一般为开关 + 这部分的导线，）如12黄色连线部分所示。对于该单元内部，根据其单位故障率接近 0，故近似认为各点的用电可靠性相同。

在用电可靠性中，采用逐层迭代的方法进行计算。若用户能够成功用电，则意味着

从电源至用户的每个电路单元都不出现故障，即用户用电可靠性为：

$$R = \Pi R_i \tag{1}$$

其中 R 为用户的用电可靠性，R_i 为电路单元的用电可靠性。

在单电源的情况下，用户的用电可靠性采用离电源由近至远依次迭代的方法进行计算。以3为例，当先计算出电路单位 1 处的用电可靠性 R_1 后，则用户 A 的可靠性可以表示为 $R_A = R_1 * R_{rest-a}$，同理，电路单位 2 的可靠性可以表示为 $R_2 = R_1 * R_{12}$，接着用户 B 的用户用电可靠性则进一步的在电路单位 2 的基础上进行计算。

由上述计算方法，本模型需要构建电路单位及用户的连接次序矩阵 W，其中：$w_{ij} = 1$ 表示由电路单位 j 连向电路单位 i，其余情况下 $w_{ij} == 0$，及每个电路单位的可靠性 (1-故障率) 向量 Z。用户的用电可靠性计算方法如下伪代码所示：

Algorithm 2 用户用电可靠性求解算法

Initialization: $W, Z, k = user_id, R = 1$
while $k! = 1$ **do**
  $R = R[:, k] * Z$
  $k' = argR[k', k] = 1$
end while
Output: R

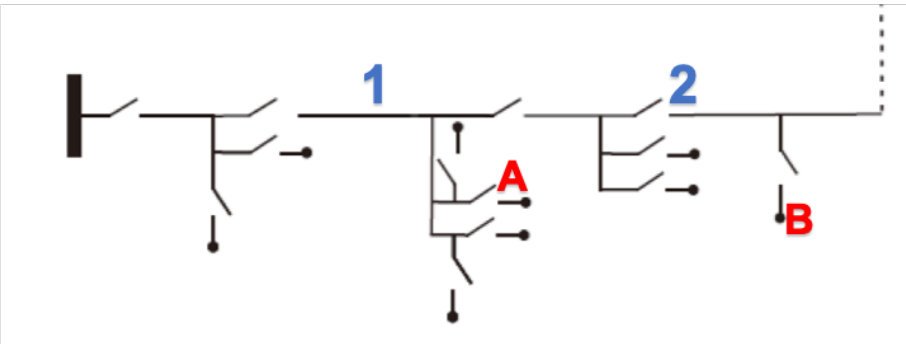

图 3: 用电可靠性计算示意

4 问题一与问题二

4.1 模型的假设

本题需要在电源坐标、用户坐标、每个用户的需求功率给定的情况下，求解满足供给功率条件下，设计建造费用最低的单供配电网。由于题目中已经要求为树状供电网，由于开关布置原则3，得知每个用户前端、分叉点上均有开关，又由于从电源引出后主

线路亦有开关，则此树状结构每条边上均有开关。优化目标——总建设费用的表达式如下：

$$F = 325.7 * F_m + L_a * 188.6 + L_b * 239.4 + 2.6 * S_1 + 56.8 * S_2 \tag{2}$$

由题意，我们将线路分为两级，分别为主线和支线，其中主线仅用于连接各支线。考虑到在线路造价最短的情况中，用于连接各用户节点和其他虚拟节点。故问题 1 的求解按照两级的假设，具体分为两步，第一步确定哪些用户处于同一支线以及确定主线的位置，第二步确定每个支线内部如何布置线路，使得优化目标 F 最小化。

4.2 模型的求解

4.2.1 主线的求解-聚类与 TSP 模型求解

根据模型的假设，我们首先对原来所有用户进行 **k-means 聚类**，找出每类的几何中心，以一类作为一个支线上的所有用户，然后利用模拟退火法求解**单向 TSP 问题**，找出一条不返回的连接电源和这些中心点的最短路径，该最短路即为所求的主线.

k-means 聚类　一种迭代求解的聚类分析算法，其步骤是随机选取 K 个对象作为初始的聚类中心，然后计算每个对象与各个种子聚类中心之间的距离，把每个对象分配给距离它最近的聚类中心。

聚类中心以及分配给它们的对象就代表一个聚类。每分配一个样本，聚类的聚类中心会根据聚类中现有的对象被重新计算。这个过程将不断重复直到满足某个终止条件。终止条件可以是没有（或最小数目）对象被重新分配给不同的聚类，没有（或最小数目）聚类中心在发生变化，误差平方和局部最小。

TSP 问题　已知 n 个城市相互之间的距离，某一旅行商从某个城市出发访问每个城市有且仅有一次，最后回到出发城市，如何安排才使其所走路线距离最短。简言之，就是寻找一条最短的遍历 n 个城市的路径，即一个自然子集 $X = 1, 2, \cdots, n$ 的排列 $X = V_1, V_2, \cdots, V_n$ 使得

$$T_d = \sum_{i=1}^{n-1} d(V_i, V_{i+1}) + d(V_n, V_1) \tag{3}$$

取得最小值。

然而，传统的 TSP 问题是一个闭环，即要回到初始起点。在本题中并不需要回到初始起点。因此我们采取的策略是，增加一个虚拟节点，该节点到所有点的距离为 0，在求解出包含该点最短路径之后，删掉与该点连接的两条边，剩下的路径即为最小单向路径。使用启发式算法中的模拟退火 (simulated annealing, SA) 算法来求解该 TSP 问题。

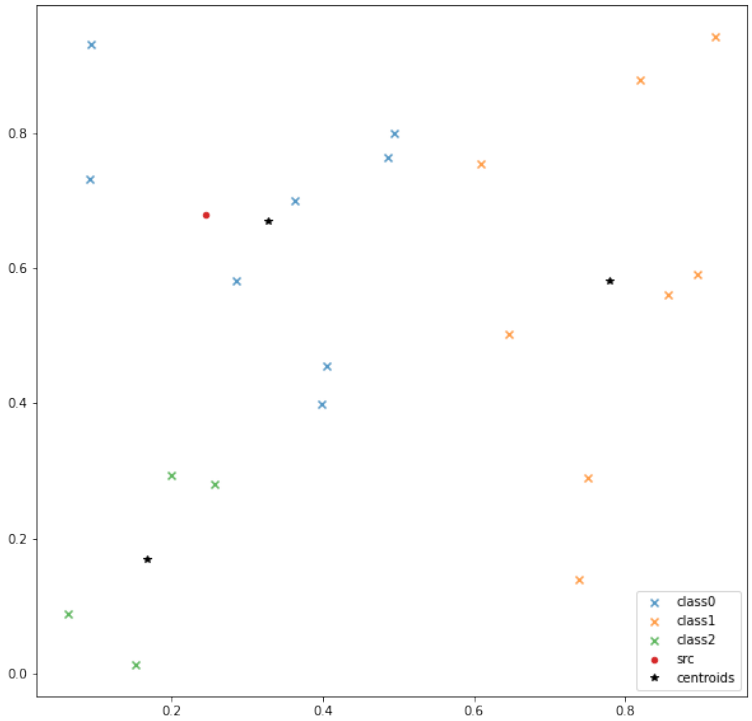

图 4: K-means 聚类后结果（以 3 类为例）

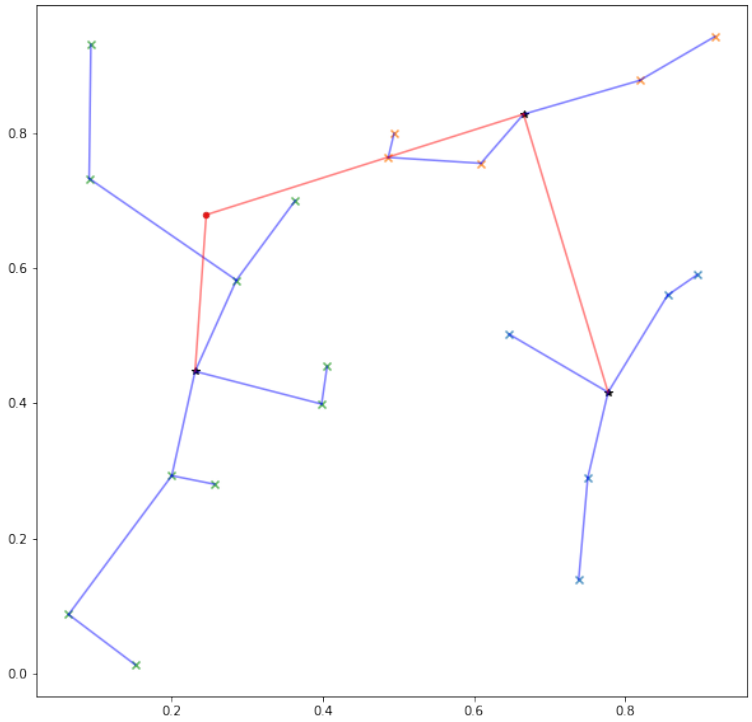

图 5: 分 3 类时加上单向主线的结果

4.2.2 主线的求解-最小二乘法

由题可知，主线的建设单价比支线建设价格高，若建设总距离长度相近，则主线长度更短为更优解，由于直线距离较折线与曲线大幅的更短，于是考虑线性回归模型，使用**最小二乘法**来确定主线的位置。

最小二乘法，是求解 (k, b) 满足下式

$$(k, b) = argmin\Sigma(Y_i - k * X_i - b)^2 \tag{4}$$

其中 Y_i, X_i 为待拟合的两组数据。从几何直观上来理解，最小二乘法的几何意义是高维空间中的一个向量在低维子空间的投影。

优势 通过最小二乘获得的主线为直线，相较聚类算法的几何中心连线来说更短，在主线的建设价格上获得较大的优势。同时，其计算复杂度相较聚类及 tsp 模型求解而言无疑简单得多。

局限性 最小二乘得到的直线可能与电源距离较远，若限制该直线通过电源，则拟合效果会受到影响。此外，最小二乘得到主线的方法受用户的分布情况影响较大，只能在线性程度较好的情况下使用，故设定拟合的 R 值作为最小二乘法和聚类算法的选择标准。

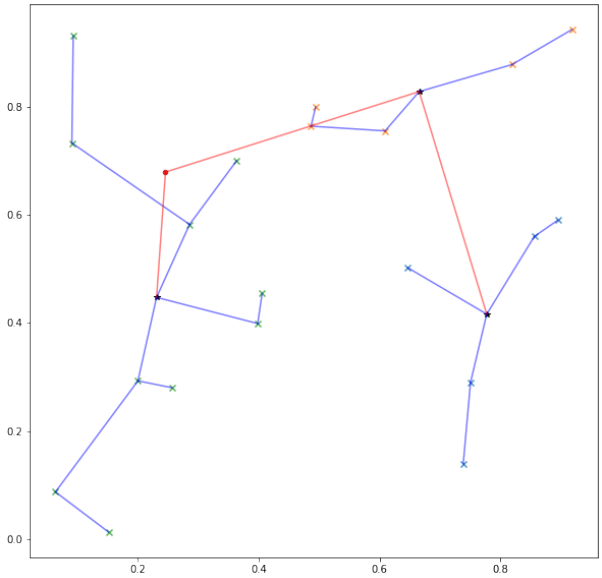

图 6: 最小二乘模型示意图

4.2.3 支线的求解

根据模型的假设，在支线的求解中，需要使用支线 A 与支线 B 两种材料与开关，将这个部分中的所有用户都与主线相连接，并使得建设价格尽可能的小。同时，由于支线 A 与支线 B 两种材料的单价不同，故仅考虑路线总和最短是不合适的。由此，可以将其视作**斯坦纳树模型**。

斯坦纳树模型，是用于解决基于给定的点集，允许在给定点外增加额外的点，使生成的最短网络开销最小的模型。该模型的求解为 NP 问题，可以使用动态规划算法进行求解，具体算法思路如下所述。

Algorithm 3 斯坦纳树模型求解算法

1: 采用动态规划算法求解
2: $f[s][i] = min\{f[t][i] + f[s-t][i]\}$
3: $f[s][i] = min_{j=1,j\neq i}\{f[s][j] + e[i][j]\}$
4: solve $g[s] = min f[s][i]$

4.2.4 问题一求解结果

我们生成的数据中包含 50 个用户，在对分类数进行 3-10 类遍历后发现分 6 类可以使得造价最低。

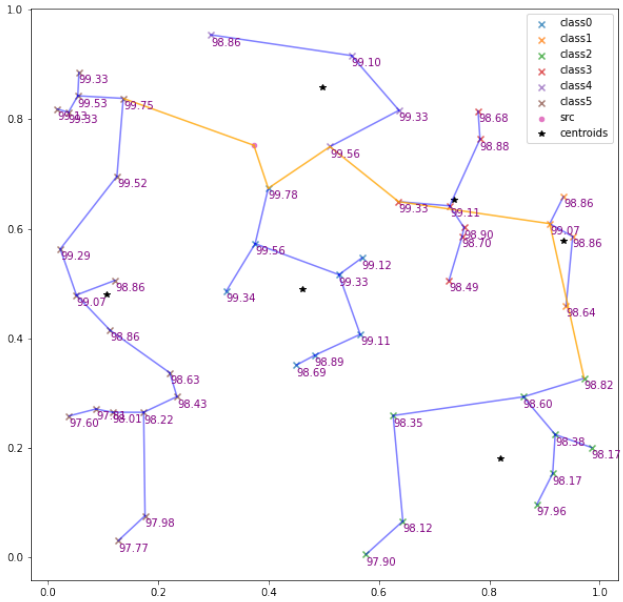

图 7: 分 6 类时加上单向主线的结果

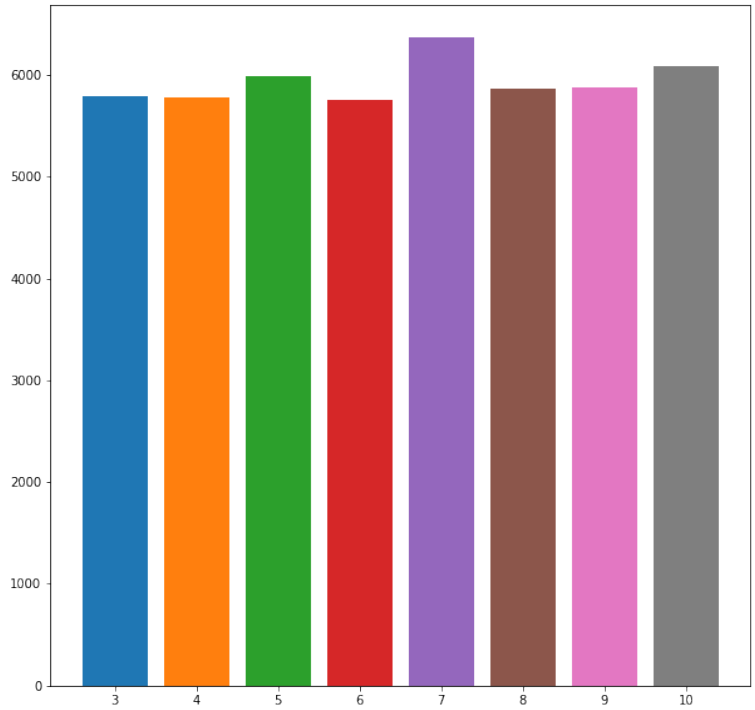

图 8: 问题一分类数目和造价之间的关系

4.2.5 问题二求解结果

对于问题二，可以在问题一的基础上将两个电源分别规划进两个不同的主线中，具体操作是先利用模拟退火求解 TSP 问题找出一条最短路径，该路径被两个电源分为两段，在两段中分别删去最长的一条边，留下的即为所求路径。

例如，分成 4 类时规划如下图所示：

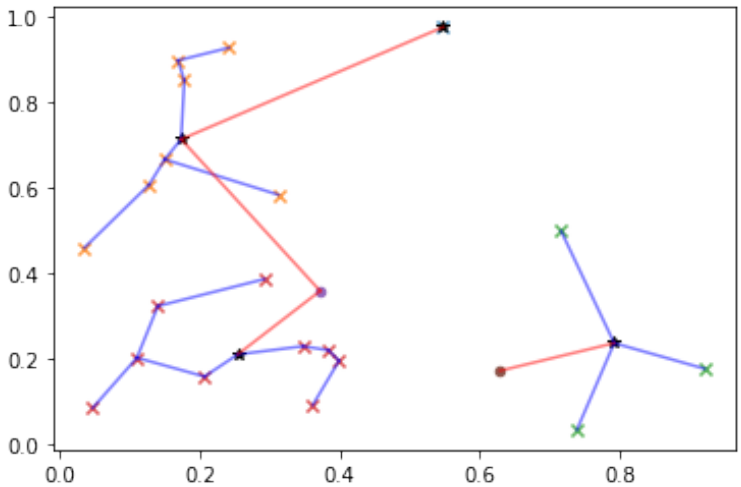

图 9: 分 4 类时加上两条主线规划结果

经过遍历求解，发现分 5 类时花费最少：

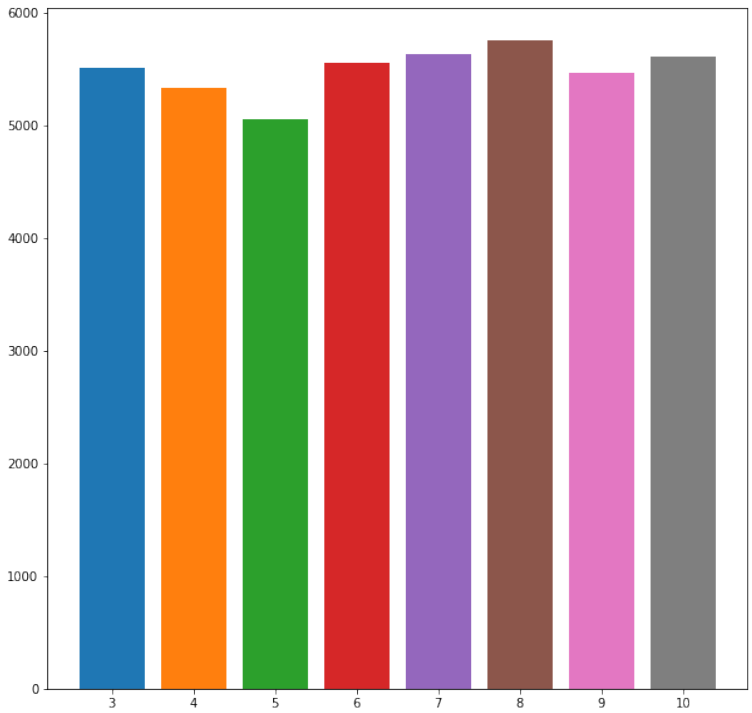

图 10: 问题二分类数目和造价之间的关系

此时路径规划及用户用电可靠性数值为:

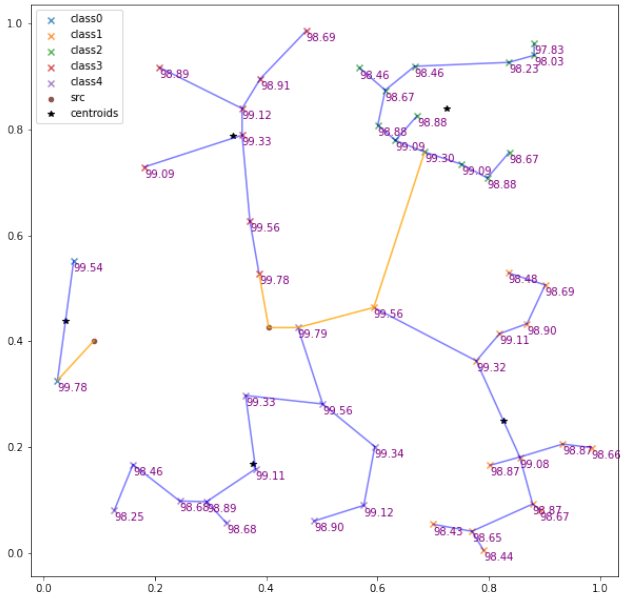

图 11: 分 5 类时加上两条主线规划结果

5 问题三与问题四

在问题三与问题四中，优化目标与限制条件为最低用户用电可靠性与总费用。问题三要求在建造总费用满足约束条件的前提下，优化目标为最低用户用电可靠性，求最大值；问题四要求在最低用户用电可靠性满足约束条件的前提下，优化目标为建造总费用，求最小值。

5.1 模型的假设

为了简化添线时的位置选择，本文将两个节点直接的部分视作一个单元（一般为开关 + 这部分的导线，）如12黄色连线部分所示。对于该单元内部，根据其单位故障率接近 0，故近似认为各点的用电可靠性相同。为计算方便，问题三与问题四选取了 20 个用户的情况下问题二的求解结果作为前置条件。

5.1.1 用电可靠性更新模型

每次更新时，双供配电网中会增加一条线，由该电线添加的位置，可以将每个电网分成两部分——比该电路单元更靠近电源的部分 P1 以及更远离电源的部分 P2.

5.1.2 基于贪婪思想的每轮需考虑用户的确定方式

一般的贪婪想法中，每轮考虑当前最低的用户用电可靠性，然后添加的线路两端中的一端便确定在该用电单位上。而本文基于此思想做了相应的改进。将用户用电可靠性从小到大排序，按照顺序逐一对两两之间的差值进行比较，若满足：

$$R[i+1] - R[i] \leq k * (R[i+2] - R[i+1]) \tag{5}$$

则说明第 i+2 个点较第 i+1 和第 i 个点来说，在阈值 k 的标准下较大，即对最低用户用电可靠性在当前情况下不会起到决定性作用，本轮不需考虑第 i+2 个点，判断终止。其中，阈值 k 的值需要根据 X 及初始用电可靠性确定。

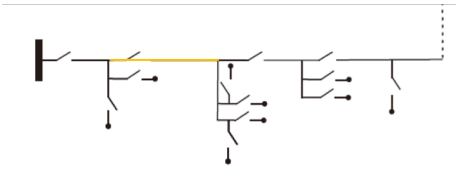

图 12: 电路单元结构示意图

Algorithm 4 添线策略

Initialization: 两个已经完成连线的单供配电网，阈值 k
将两个单供配电网分别单元化，计算出每个单元当前的用电可靠性
while Fare < X **do**
    将两个电网中用户的用电可靠性升序排序，得到电网 1 对应的数组 $R_1[n1]$ 及电网 2 对应的数组 $R_2[n2]$.
    Initialization: i = 1
    % 判断当前考虑的目标数组（即最小值所在的数组）
    if $R_1[1] < R_2[1]$ **then**
        $R[1:n1] = R_1[1:n1]$
    else
        $R[1:n2] = R_2[1:n2]$
    end if
    确定本轮考虑的用户及数量，数量记为 n
    根据本轮需要考虑的用户，确定这若干个点包含的区域 D, 及电路单元集合 E_j.
    对每个电路单元 E_j，在另一个电路中遍历所有电路单元，找出在这两点添线后最低可靠性提升最大的位置。
    对每个区域内所有的电路单元 E_j，得到最低可靠性提升最大的添线方式，将该电线添加至原配电网络。
    $Fare = Fare + L * 325.7 + 56.8$ % 计算价格，更新 Fare
    应用用户用电可靠性更新算法，更新 $R_1[n]$ 和 $R_2[n]$
end while

Output: 完成添线的双供配电网

对于第四题，首先需要将循环终止条件改为所有点的可靠度均大于 Y%，接着在每轮需考虑用户的确定方式中，对需考虑的用户范围增加初始用电可靠性小于 Y% 的条件。

5.2 求解结果

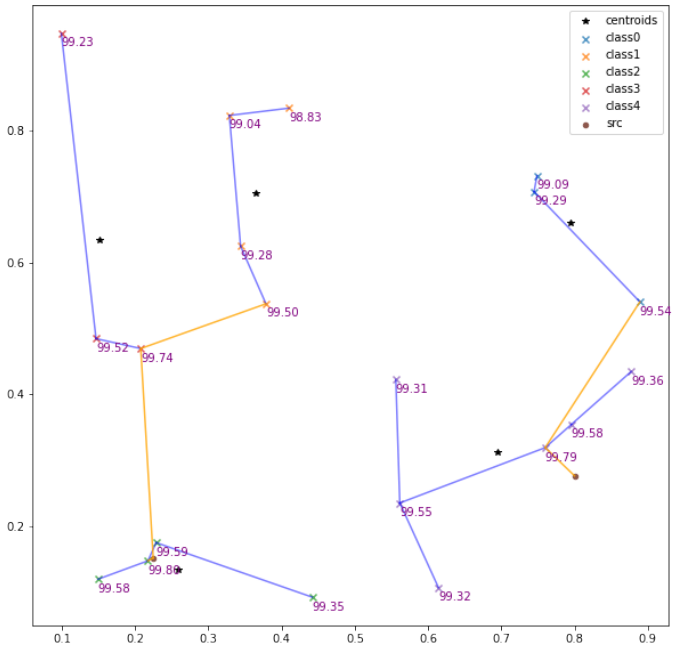

图 13: 问题二求解结果

此时，$cost = 1128.48k$。

对于问题三，设定 X = 1500k。

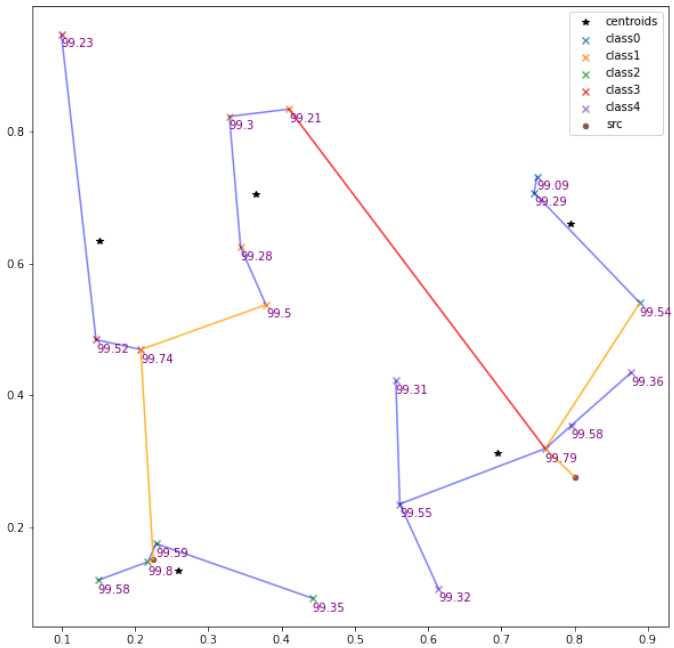

图 14: 问题三第一步结果

此时，$cost = 1333.98k$

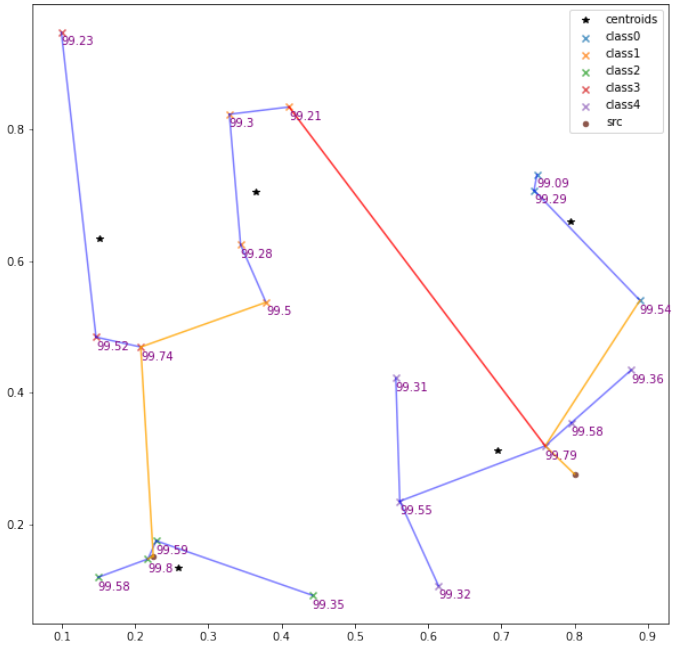

图 15: 问题三第二步结果

此时，$cost = 1468.17k$. 由于再添加下一根电线时，cost 大于 X，故第二步结果即为最终结果。

对于问题四，设定 Y = 99%。

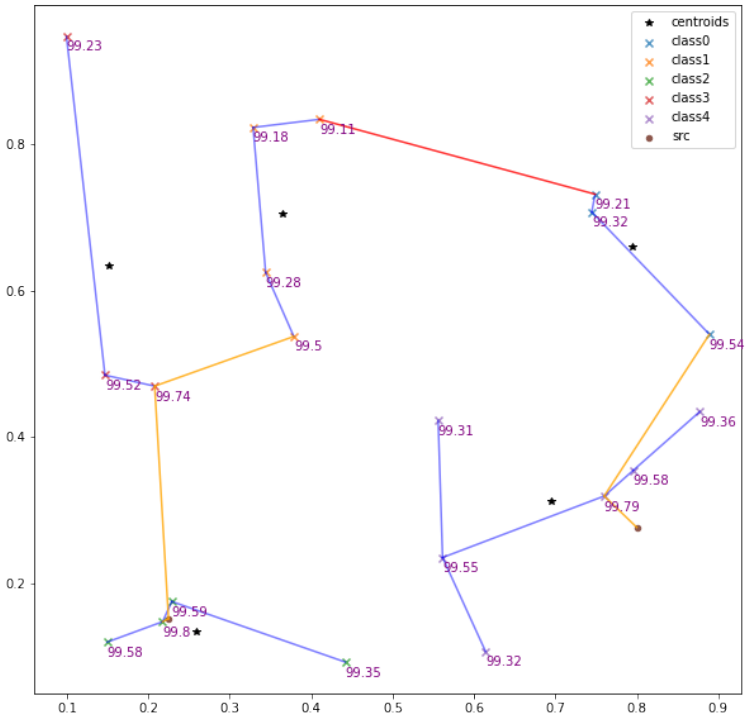

图 16: 问题四第一步结果

此时 $cost = 1246.373k$，发现已满足结果，故第一步结果即为最终结果。

6 参考文献

[1] 肖峻, 崔艳妍, 王建民, 等. 配电网规划的综合评价指标体系与方法 [J]. 电力系统自动化, 2008, 32(15):5.

[2] 倪秋龙, 黄民翔. 基于支路交换的模拟退火算法在配电网规划中的应用 [J]. 电力系统及其自动化学报, 2000, 12(4):5.

[3] 肖峻, 李振生, 张跃. 基于最大供电能力的智能配电网规划与运行新思路 [J]. 电力系统自动化, 2012(13).

[4] 胡蓉, 张焰, 范超, 等. 配电网规划后评估指标体系的研究 [J]. 华东电力, 2007, 35(008):70-74.

[5] 沈欣炜, 朱守真, 郑竞宏, 等. 考虑分布式电源及储能配合的主动配电网规划-运行联合优化 [J]. 电网技术, 2015, 39(7):8.

[6] 李子韵, 陈楷, 龙禹, 等. 可靠性成本/效益精益化方法在配电网规划中的应用 [J]. 电力系统自动化, 2012, 36(11):5.

[7] 莫志伟. 城市配电网规划探讨 [J]. 城市建设理论研究: 电子版, 2011, 000(020):1-3.

[8] 范明天. 基于可靠性的配电网规划思路和方法讲座一基于可靠性的配电网规划的概况和基本理念 [J]. 供用电, 2011, 28(1):4.

[9] 吴思谋, 蔡秀雯, 王海亮. 面向供电可靠性的配电网规划方法与实践应用 [J]. 电力系统及其自动化学报, 2014, 26(6):6.

[10] 陈光华, 杜栋, 庞庆华, 等. 基于可靠性的配电网规划评估体系 [J]. 电力科学与技术学报, 2013, 28(1):7.

7 附录

7.1 pro1.py

```
# %%
import numpy as np
import matplotlib.pyplot as plt
# import scipy
# plt.rcParams['font.sans-serif'] = ['SimHei']
# plt.rcParams['axes.unicode_minus'] = False

# %%
N_class=3
N_points=20
ip=np.random.random((N_points,2))
source=np.random.random((1,2))
ip.shape
source


# %%
plt.figure(figsize=(10,10))
plt.scatter(ip[:,0],ip[:,1],marker='x',label='des')
plt.scatter(source[:,0],source[:,1],marker='o',label='src',s=20)
plt.legend()
plt.show()

# %%


# %%
from scipy.cluster.vq import kmeans2
centroid, label = kmeans2(ip, N_class, minit='points')
counts = np.bincount(label)
plt.figure(figsize=(10,10))
for i in range(N_class):
    plt.scatter(ip[label == i][:,0],ip[label == i][:,1],marker='x',
        alpha=0.75,label='class'+str(i))
plt.scatter(source[:,0],source[:,1],marker='o',label='src',s=20)
plt.plot(centroid[:, 0], centroid[:, 1], 'k*', label='centroids')
```

```
# plt.axis('equal')
plt.legend()
plt.show()

# %%
# 想法1
# 可以证明几时可以增加虚拟节点，最终的结果在路径最短时仍然是一个最小生成树

# 采用迭代的方法不断聚类分组
# lasr
# while(1):

#     for i in range()

# %%
from scipy.sparse import csr_matrix
from scipy.sparse.csgraph import minimum_spanning_tree
X = csr_matrix([[0, 8, 0, 3],
                [0, 0, 2, 5],
                [0, 0, 0, 6],
                [0, 0, 0, 0]])
Tcsr = minimum_spanning_tree(X)
# Tcsr.toarray()

# %%
from scipy.spatial.distance import cdist
def get_dense(points):
    return cdist(points,points,metric='euclidean')

def dist(a,b):
    return np.sqrt((a[0]-b[0])**2+(a[1]-b[1])**2)


# %%
from sko.SA import SA_TSP

# %%
import sklearn

```

```python
# %%
# import

# %%
# 想法2
# 主线就一条  一个发电站牵出一条主线
import scipy.sparse as sp
import time

# 添加一个节点真的会减少吗?
best_centroid=centroid
best_label=label
best_counts =counts
best_cost=np.Inf
plt.figure(figsize=(10,10))
# for class_num in range(3,N_points):
cost_=list(np.zeros(N_points+1))
for class_num in range(3,11):
    centroid, label = kmeans2(ip, class_num, minit='points')
    counts = np.bincount(label)
    for i in range(class_num):
        plt.scatter(ip[label == i][:,0],ip[label == i][:,1],marker='x',alpha=0.75,label='class'+str(i))
    plt.scatter(source[:,0],source[:,1],marker='o',label='src',s=20)
    plt.plot(centroid[:, 0], centroid[:, 1], 'k*', label='centroids')
    forest=[]
    cost=0
    for tree_num in range(class_num):
        nodes=ip[label == tree_num]
        nodes=np.append(nodes,centroid[tree_num,:].reshape(1,2),axis=0)
        # if(not tree_num) :print(nodes)
        Tcsr=minimum_spanning_tree(sp.csr_matrix(get_dense(nodes)))
        tree=Tcsr.toarray()
        # print(tree)
        for i in range(nodes.shape[0]):
            for j in range(nodes.shape[0]):
                if(tree[i][j]):
                    cost+=3*478.4*tree[i][j]
                    # print(nodes[[i,j],0],nodes[[i,j],1])
```

```
                    plt.plot(nodes[[i,j],0],nodes[[i,j],1],c='blue',alpha=0.5)
        print(tree)
    # for tree_num那个
    # TSP求解最短路径，由于不需要回来，引入一个虚拟点，该点到所有点距离为0，最后删掉

    num_points = class_num+2                #num_points指需要遍历的节点个数
    points_coordinate = centroid.copy()
    points_coordinate=np.append(points_coordinate,source.reshape(1,2),axis=0)
    length = points_coordinate.shape[0]
    dist_matrix = np.zeros([length+1, length+1])
    for i1 in range(length):
        for j1 in range(length):
            dist_matrix[i1][j1] = np.sqrt(
                (points_coordinate[i1][0]-points_coordinate[j1][0])**2+(points_coordinate[i1][1]-points_coordinate[j1][1])**2)
    length=length+1
    print(dist_matrix)
    sol_new = list(range(1, length+1))
    sol_current = sol_new
    sol_best = sol_new
    E_current = 100000
    E_best = 100000
    t0 = 97
    tf = 3
    t = t0
    a = 0.99
    markov_length = 500*class_num
    while t >= tf:
        for r in range(1, markov_length+1):
            if length >= 4:
                if np.random.rand() < 0.5:
                    ind1, ind2 = 0, 0
                    while ind1 == ind2:
                        ind1 = int(np.floor(np.random.rand()*length))
                        ind2 = int(np.floor(np.random.rand()*length))
```

```
                tmp1 = sol_new[ind1]
                sol_new[ind1] = sol_new[ind2]
                sol_new[ind2] = tmp1
            else:
                ind1, ind2, ind3 = 0, 0, 0
                while ind1 == ind2 or ind1 == ind3 or ind2 == ind3:
                    ind1 = int(np.floor(np.random.rand()*length))
                    ind2 = int(np.floor(np.random.rand()*length))
                    ind3 = int(np.floor(np.random.rand()*length))
                tmp1, tmp2, tmp3 = ind1, ind2, ind3
                if ind1 < ind2 and ind2 < ind3:
                    pass
                elif ind1 < ind3 and ind3 < ind2:
                    ind2, ind3 = tmp3, tmp2
                elif ind2 < ind1 and ind1 < ind3:
                    ind1, ind2 = tmp2, tmp1
                elif ind2 < ind3 and ind3 < ind1:
                    ind1, ind2, ind3 = tmp2, tmp3, tmp1
                elif ind3 < ind1 and ind1 < ind2:
                    ind1, ind2, ind3 = tmp3, tmp1, tmp2
                elif ind3 < ind2 and ind2 < ind1:
                    ind1, ind2, ind3 = tmp3, tmp2, tmp1
                tmplist1 = sol_new[ind1:ind2]
                sol_new[ind1:ind1+ind3-ind2] = sol_new[ind2:ind3]
                sol_new[ind1+ind3-ind2:ind3] = tmplist1
        else:
            sol_new = list(range(1, length+1))
        E_new = 0
        for i2 in range(0,length-1):
            E_new = E_new+dist_matrix[sol_new[i2]-1][sol_new[i2+1]-1]
        E_new = E_new+dist_matrix[sol_new[length-1]-1][sol_new[0]-1]
        if E_new < E_current:
            E_current = E_new
            sol_current = sol_new.copy()
            if E_new < E_best:
                E_best = E_new
                sol_best = sol_new.copy()
```

```
            else:
                if np.random.rand() < np.exp(-(E_new-E_current)/t):
                    E_current = E_new
                    sol_current = sol_new.copy()
                else:
                    sol_new = sol_current.copy()
        t = t*a

    # print(E_best)
    # print(sol_best)
    for i in range(0,length-1):
        if sol_best[i] != length and sol_best[i+1] !=length:
            plt.plot(points_coordinate[[sol_best[i]-1,sol_best[i+1]-1],0],points_coordinate[[sol_best[i]-1,sol_best[i+1]-1],1],c='r',alpha=0.5)
            cost=cost+2*651.4*dist(points_coordinate[sol_best[i]-1],points_coordinate[sol_best[i+1]-1])
    if sol_best[length-1]!=length and sol_best[0]!=length:
        plt.plot(points_coordinate[[sol_best[length-1]-1,sol_best[0]-1],0],points_coordinate[[sol_best[length-1]-1,sol_best[0]-1],1],c='r',alpha=0.5)
        cost=cost+3*651.4*dist(points_coordinate[sol_best[length-1]-1],points_coordinate[sol_best[0]-1])
    cost=cost+56.8*class_num
    cost_[class_num]=cost
    print(cost)
    plt.figure(figsize=(10,10))
    plt.show()
    plt.clf()
print(cost_)


# %%
plt.figure(figsize=(10,10))
for i in range(3,11):
    plt.bar(i,cost_[i])
plt.show()


# %%
```

```
s_A[:,0]

# %%
```